TNOps checks: abelian block-sparse tensors against dense arrays

`TNOps` provides tagged legs with arrows, block tensors of any rank with a flux, sliced tensors, block-wise contractions, fusion of legs, factorizations, a global degeneracy-safe cut and fermionic parity (conventions in `TNOps/README.md`). This notebook runs one section per group of checks of the refactor sheet (`mpNRG/docs/TNOPS_REFACTOR.md`, Sec. 3.7) and prints the achieved accuracies; the unit tests (`julia --project=TNOps/test TNOps/test/runtests.jl`) assert the same checks at 10^{-13}.

Block rule. A block with sectors $(s_1, \ldots, s_N)$ of a tensor with flux Δ exists only if $\bigoplus_{\mathrm{Out}} \mathrm{label}(s_i) = \bigoplus_{\mathrm{In}} \mathrm{label}(s_i) \oplus \Delta$.

Every operation is compared with the same operation on `todense` arrays, for random tensors of rank 1–5 with 2–5 sectors per leg, random fluxes, the symmetries U1×U1×Z2, Z3 (×Pf) and parity only, real and complex elements.

In [1]:
```
using TNOps, LinearAlgebra, Random, Printf
# IJulia's `In`/`Out` (the cell history) shadow the arrows exported by
TNOps: write TNOps.In / TNOps.Out
import TNOps: legsectors, blockkeys, allowed_keys, keyflux, isallowed,
legs, leg, flux, nblockrows
println("Julia ", VERSION, ", threads ", Threads.nthreads(), ", TNOps ",
pkgversion(TNOps))
const GROUPS = [
    ("U1×U1×Z2", SymmetryGroup(U1(:N), U1(:twoSz)), rng -> (n = rand(rng,
-2:2); (n, rand(rng, -2:2), mod(n, 2)))),
    ("Z3", SymmetryGroup(Zn(:C, 3)), rng -> (rand(rng, 0:2), rand(rng,
0:1))),
    ("parity", SymmetryGroup(), rng -> (rand(rng, 0:1),)),
]
function randleg(rng, G, gen; nsec = rand(rng, 2:5), dmax = 3, arrow =
rand(rng, (TNOps.In, TNOps.Out)), tagged = rand(rng) < 0.3)
    secs = unique!([LegSector(sector(G, gen(rng)), tagged ? rand(rng, 0:2)
: 0) for _ in 1:(4nsec)])[1:min(nsec, end)]
    return Leg(secs, [rand(rng, 1:dmax) for _ in secs]; arrow = arrow)
end
randflux(rng, ls) = keyflux(ls, ntuple(i -> rand(rng, legsectors(ls[i])),
length(ls)))
relerr(a, b) = norm(a - b) / max(norm(b), 1e-300)
RESULTS = Dict{String,Float64}()
record!(k, v) = (RESULTS[k] = max(get(RESULTS, k, 0.0), v))
```

```
Julia 1.13.0, threads 4, TNOps 0.1.0
```

Out[1]:
```
record! (generic function with 1 method)
```

1. Construction, block rule, dense round trip, arithmetic, permutation

`fromdense(todense(A)) == A`; forbidden blocks are never stored; `+`, `-`, scalar `*`, `conj`, `norm`, `dot` and the bosonic `permute` agree with the dense operations.

In [2]:
```
nforbid = 0; ntensors = 0
for (gname, G, gen) in GROUPS
    rng = MersenneTwister(sum(codeunits(gname)))
    for N in 1:5, T in (Float64, ComplexF64), rep in 1:3
        ls = ntuple(_ -> randleg(rng, G, gen; dmax = N ≤ 3 ? 3 : 2), N)
        Δ = randflux(rng, ls)
        A = randtensor(rng, T, ls, Δ; density = 0.8); B = randtensor(rng,
T, ls, Δ; density = 0.8)
        global ntensors += 1
        global nforbid += count(k -> keyflux(ls, k) != Δ, blockkeys(A))
        D = todense(A); DB = todense(B)
        record!("dense round trip", norm(todense(fromdense(D, ls, Δ)) - D))
        record!("arithmetic", relerr(todense(A - 2.5 * B), D - 2.5 * DB))
        record!("arithmetic", relerr(todense(conj(A)), conj(D)))
        record!("arithmetic", abs(norm(A) - norm(D)) / max(norm(D), 1e-
300))
        record!("arithmetic", abs(dot(A, B) - dot(D, DB)) / max(1, norm(D)
* norm(DB)))
        p = Tuple(randperm(rng, N))
        record!("permute", norm(todense(permute(A, p)) - permutedims(D,
p)))
    end
end
@printf("%d random tensors (ranks 1-5, 3 symmetries, real and complex);
stored forbidden blocks: %d\n", ntensors, nforbid)
for k in ("dense round trip", "arithmetic", "permute")
    @printf("%-18s max deviation %.1e\n", k, RESULTS[k])
end
```

```
90 random tensors (ranks 1-5, 3 symmetries, real and complex); stored forbid
den blocks: 0
dense round trip   max deviation 0.0e+00
arithmetic         max deviation 4.0e-16
permute            max deviation 0.0e+00
```

2. `contract`, `outer`, `tr` and `ncon` against explicit dense loops

The dense reference is a plain loop over all index values (no reshape tricks). Mismatched legs raise errors that name the legs.

In [3]:
```
function dense_contract(DA, ia, DB, ib)
    oa = [i for i in 1:ndims(DA) if !(i in ia)]; ob = [i for i in
1:ndims(DB) if !(i in ib)]
```

```
    C = zeros(promote_type(eltype(DA), eltype(DB)), (size(DA)[oa]...,
size(DB)[ob]...))
    for Ia in CartesianIndices(DA), Jb in CartesianIndices(size(DB)[ob])
        bi = zeros(Int, ndims(DB))
        for (x, y) in zip(ia, ib); bi[y] = Ia[x]; end
        for (m, y) in enumerate(ob); bi[y] = Jb[m]; end
        C[Tuple(Ia)[oa]..., Tuple(Jb)...] += DA[Ia] * DB[bi...]
    end
    return C
end
ncontr = 0
for (gname, G, gen) in GROUPS
    rng = MersenneTwister(7 + sum(codeunits(gname)))
    for trial in 1:15, T in (Float64, ComplexF64)
        NA = rand(rng, 1:4); NB = rand(rng, 1:4); nc = rand(rng, 0:min(NA,
NB, 2))
        lsA = ntuple(_ -> randleg(rng, G, gen; dmax = 2), NA)
        ia = Tuple(randperm(rng, NA)[1:nc]); ib = Tuple(randperm(rng, NB)
[1:nc])
        lsB = Vector{Any}(undef, NB)
        for (x, y) in zip(ia, ib); lsB[y] = dual(lsA[x]); end
        for y in 1:NB; isassigned(lsB, y) || (lsB[y] = randleg(rng, G, gen;
dmax = 2)); end
        A = randtensor(rng, T, lsA, randflux(rng, lsA)); B =
randtensor(rng, Float64, Tuple(lsB), randflux(rng, Tuple(lsB)))
        C = contract(A, ia, B, ib); Cd = dense_contract(todense(A), ia,
todense(B), ib)
        global ncontr += 1
        record!("contract", norm(todense(C) - Cd) / max(1, norm(Cd)))
        record!("forbidden output blocks", count(k -> !isallowed(C, k),
blockkeys(C)))
    end
    # ncon: X(a,b,c) Y(b*,d) W(c*,d*) → vector on a, contraction order (3,
1, 2)
    rng = MersenneTwister(11 + sum(codeunits(gname)))
    a, b, c, d = (randleg(rng, G, gen; dmax = 2) for _ in 1:4)
    X = randtensor(rng, ComplexF64, (a, b, c), randflux(rng, (a, b, c)))
    Y = randtensor(rng, ComplexF64, (dual(b), d), randflux(rng, (dual(b),
d)))
    W = randtensor(rng, ComplexF64, (dual(c), dual(d)), randflux(rng,
(dual(c), dual(d))))
    R = ncon([X, Y, W], [[-1, 1, 2], [1, 3], [2, 3]]; order = [3, 1, 2])
    Dx, Dy, Dw = todense(X), todense(Y), todense(W)
    ref = [sum(Dx[i, j, k] * Dy[j, m] * Dw[k, m] for j in axes(Dx, 2), k in
axes(Dx, 3), m in axes(Dy, 2)) for i in axes(Dx, 1)]
    record!("ncon", norm(todense(R) - ref) / max(1, norm(ref)))
    # traces
    l = randleg(rng, G, gen; dmax = 3, arrow = TNOps.Out)
    H = randtensor(rng, ComplexF64, (l, dual(l)), neutral(labeltype(G)))
    record!("tr", abs(tr(H) - tr(todense(H))) / max(1, norm(todense(H))))
end
l1 = Leg(Dict(sector(SymmetryGroup(U1(:N)), (0, 0)) => 2); name = :left)
Ax = BlockTensor{Float64}((l1, dual(l1)),
neutral(labeltype(SymmetryGroup(U1(:N)))))
msg = try contract(Ax, (1,), Ax, (1,)); "no error" catch e; e.msg end
@printf("%d random contractions (ranks 1-4, 0-2 contracted pairs): max
```

```
relative deviation %.1e; forbidden output blocks %d\n",
        ncontr, RESULTS["contract"], Int(RESULTS["forbidden output
blocks"]))
@printf("ncon (three tensors, order 3,1,2): %.1e;  tr: %.1e\n",
RESULTS["ncon"], RESULTS["tr"])
println("duality error: ", msg)
```

```
90 random contractions (ranks 1-4, 0-2 contracted pairs): max relative devia
tion 3.1e-16; forbidden output blocks 0
ncon (three tensors, order 3,1,2): 2.4e-16;  tr: 1.0e-16
duality error: cannot contract leg 1 :left [Out; Sector(0, 0)=>2] of the fir
st tensor with leg 1 :left [Out; Sector(0, 0)=>2] of the second: not dual
```

3. Fusion: `fuse_legs ∘ split_legs = 1`, and the frozen output of the pre-refactor fusion map

The fusion layout is the one of the pre-refactor NRG product basis: pairs ordered by outer then inner sector, product index `offset + (iσ-1)·dq + s`. `fusionmap` and `kron_fused` are compared **exactly** with the output of the old `fusionmap` and `kron_op`, frozen at the tag `pre-tensor-refactor` (fixtures written by `mpNRG/test/baseline/make_baseline.jl`: the fusion maps of every shell of a SIAM run and the three terms of H_2).

In [4]:
```
include(joinpath(@__DIR__, "..", "test", "fixtures",
"old_layer_fixtures.jl"))
lab(M, t) = Sector{M,length(M)}(t)
legof(M, sp) = Leg(Dict(LegSector(lab(M, t)) => d for (t, d) in sp))
function opof(M, d)
    A = BlockTensor{Float64}(legof(M, d["row"]), legof(M, d["col"]), lab(M,
d["shift"]))
    for (t, B) in d["blocks"]; setblock!(A, lab(M, t), B); end
    A
end
M = FX_MODULI
npairs = 0; pairs_equal = true
for (n, fx) in FX_FUSION
    F = fusionmap(legof(M, fx["loc"]), legof(M, fx["prev"]))
    for (t, prs) in fx["pairs"]
        mine = [(p.σ.label.q, p.q.label.q, p.offset, p.dσ, p.dq) for p in
F.pairs[LegSector(lab(M, t))]]
        global pairs_equal &= mine == prs; global npairs += length(prs)
    end
end
nblk = 0; blocks_equal = true
for (name, fx) in FX_KRON
    X = opof(M, fx["X"]); Y = opof(M, fx["Y"])
    F = fusionmap(legof(M, fx["fusion"]["loc"]), legof(M, fx["fusion"]
["prev"]))
    R = kron_fused(X, Y, F, F); old = opof(M, fx["out"])
    global blocks_equal &= Set(blockkeys(R)) == Set(blockkeys(old)) &&
all(getblock(R, k) == getblock(old, k) for k in blockkeys(old))
    global nblk += length(blockkeys(old))
```

```
end
@printf("fusionmap: %d pairs of %d shells identical (order and offsets):
%s\n", npairs, length(FX_FUSION), pairs_equal)
@printf("kron_fused: %d blocks of 3 Kronecker products bit-identical to
kron_op: %s\n", nblk, blocks_equal)
for (gname, G, gen) in GROUPS
    rng = MersenneTwister(sum(codeunits(gname)) + 1)
    for trial in 1:8
        N = rand(rng, 2:4); ar = rand(rng, (TNOps.In, TNOps.Out))
        ls = ntuple(i -> randleg(rng, G, gen; arrow = i ≤ 2 ? ar :
rand(rng, (TNOps.In, TNOps.Out))), N)
        A = randtensor(rng, ComplexF64, ls, randflux(rng, ls))
        B, F = fuse_legs(A, (1, 2))
        record!("fuse ∘ split", norm(todense(split_legs(B, 1, F) - A)))
    end
end
@printf("split_legs(fuse_legs(A)) - A: %.1e (24 random tensors)\n",
RESULTS["fuse ∘ split"])
```

```
fusionmap: 252 pairs of 6 shells identical (order and offsets): true
kron_fused: 54 blocks of 3 Kronecker products bit-identical to kron_op: true
split_legs(fuse_legs(A)) - A: 0.0e+00 (24 random tensors)
```

4. Factorizations reconstruct the tensor

`eigen` (hermitian, neutral flux; rank 2 and a rank-4 map $(a, b) \to (a, b)$), `svd` and `qr` (rank 2 with a flux and a rank-3 tensor split as $(r, a) \mid c$). Factorizations need operator form: row legs `Out`, column legs `In`.

In [5]:
```
diagof(U, vals, i) = (Λ = BlockTensor{Float64}((dual(leg(U, i)), leg(U,
i)), neutral(sectortype(U)));
                      for (s, v) in vals; isempty(v) || setblock!(Λ, (s,
s), Matrix(Diagonal(v))); end; Λ)
sectortype(A) = typeof(flux(A))
for (gname, G, gen) in GROUPS
    rng = MersenneTwister(sum(codeunits(gname)) + 2); S = labeltype(G)
    for T in (Float64, ComplexF64)
        l = randleg(rng, G, gen; dmax = 4, arrow = TNOps.Out)          #
operator form: rows Out, columns In
        X = randtensor(rng, T, (l, dual(l)), neutral(S)); H = X + X'
        vals, U = eigen(H)
        record!("eigen", relerr(todense(U * diagof(U, vals, 2) * U'),
todense(H)))
        a = randleg(rng, G, gen; dmax = 2, arrow = TNOps.Out); b =
randleg(rng, G, gen; dmax = 2, arrow = TNOps.Out)
        Y = randtensor(rng, T, (a, b, dual(a), dual(b)), neutral(S)); H4 =
Y + adjoint(Y, (1, 2), (3, 4))
        v4, U4 = eigen(H4, (1, 2), (3, 4))
        M4 = reshape(todense(H4), nblockrows(a) * nblockrows(b), :)
        record!("eigen (partition)", norm(sort(reduce(vcat, values(v4))) -
eigvals(Hermitian(M4))) / max(1, norm(M4)))
        r = randleg(rng, G, gen; dmax = 3, arrow = TNOps.Out); c =
randleg(rng, G, gen; dmax = 3, arrow = TNOps.In)
        A = randtensor(rng, T, (r, c), randflux(rng, (r, c)))
```

```
        Uu, s, V = svd(A)
        record!("svd", relerr(todense(Uu * diagof(Uu, s, 2) * V'),
todense(A)))
        Q, R = qr(A)
        record!("qr", relerr(todense(Q * R), todense(A)))
        B3 = randtensor(rng, T, (r, a, c), randflux(rng, (r, a, c)))
        U3, s3, V3 = svd(B3, (1, 2), (3,))
        record!("svd (partition)", relerr(todense(contract(contract(U3,
(3,), diagof(U3, s3, 3), (1,)), (3,), V3', (1,))), todense(B3)))
        Q3, R3 = qr(B3, (1, 2), (3,))
        record!("qr (partition)", relerr(todense(contract(Q3, (3,), R3,
(1,))), todense(B3)))
    end
end
for k in ("eigen", "eigen (partition)", "svd", "svd (partition)", "qr", "qr
(partition)")
    @printf("%-18s reconstruction %.1e\n", k, RESULTS[k])
end
```

```
eigen              reconstruction 2.4e-15
eigen (partition)  reconstruction 1.9e-15
svd                reconstruction 6.7e-16
svd (partition)    reconstruction 2.6e-15
qr                 reconstruction 2.6e-16
qr (partition)     reconstruction 3.5e-16
```

5. `global_cut` gives the kept counts of the old `truncate_shell`

The spectra of every shell of the four baseline runs of the refactor (SIAM, with and without truncation, U1×U1×Z2 and parity only, with a field), frozen with their kept counts and truncation records at the tag `pre-tensor-refactor`. `global_cut` gets `gaptol·ω`; the old record stores the gap divided by ω.

In [6]:
```
ncut = 0; same_counts = true; same_records = true
for fx in FX_TRUNCATE
    fx["last"] && continue
    M = fx["moduli"]; ω = fx["omega"]
    E = Dict(LegSector(lab(M, t)) => Float64.(e) for (t, e) in fx["E"])
    kept, rec = global_cut(E; nkeep = fx["Nkeep"], gaptol = fx["gaptol"] *
ω)
    global ncut += 1
    global same_counts &= all(kept[LegSector(lab(M, t))] == v for (t, v) in
fx["nk"])
    r = fx["rec"]
    global same_records &= (rec.requested, rec.actual, rec.cut_moved,
rec.total) == (r["requested"], r["actual"], r["cut_moved"], r["total"]) &&
                          ((isnan(rec.gap) && isnan(r["gap"])) || rec.gap
/ ω == r["gap"])
end
@printf("%d spectra: kept counts identical %s, truncation records identical
%s\n", ncut, same_counts, same_records)
```

```
G = SymmetryGroup(U1(:N))
E = Dict(LegSector(sector(G, (0, 0))) => [0.0, 1.0, 2.0],
LegSector(sector(G, (1, 1))) => [1.0 + 1e-12, 3.0])
kept, rec = global_cut(E; nkeep = 2, gaptol = 1e-8)
println("a doublet split over two sectors, nkeep = 2: kept ", sort([v for v
in values(kept)]), ", cut moved: ", rec.cut_moved)
```

```
20 spectra: kept counts identical true, truncation records identical true
a doublet split over two sectors, nkeep = 2: kept [1, 2], cut moved: true
```

6. Parity

`apply_parity(A, i)` multiplies the blocks by $(-1)^p$ of their sector on leg `i`; it equals multiplication by the dense parity matrix, which is `todense(parity_tensor(leg))`.

In [7]:
```
for (gname, G, gen) in GROUPS
    rng = MersenneTwister(sum(codeunits(gname)) + 3)
    l = randleg(rng, G, gen; dmax = 3); m = randleg(rng, G, gen; dmax = 2)
    A = randtensor(rng, ComplexF64, (l, m), randflux(rng, (l, m)))
    Zd = Diagonal([isodd(parity(s.label)) ? -1.0 : 1.0 for (s, d) in
zip(legsectors(l), l.dims) for _ in 1:d])
    record!("parity_tensor", norm(todense(parity_tensor(l)) - Zd))
    record!("apply_parity", norm(todense(apply_parity(A, 1)) - Zd *
todense(A)))
    record!("apply_parity", norm(todense(apply_parity(A, 2)) - todense(A) *
todense(parity_tensor(m))))
end
@printf("parity_tensor vs dense: %.1e;  apply_parity vs dense: %.1e\n",
RESULTS["parity_tensor"], RESULTS["apply_parity"])
```

```
parity_tensor vs dense: 0.0e+00;  apply_parity vs dense: 0.0e+00
```

7. Usage example

Build legs, fuse them, diagonalize a hermitian operator on the product per label, keep six states with `global_cut` (the cut never splits a degenerate multiplet), and split the product leg of the eigenvectors back into a rank-3 isometry.

In [8]:
```
G = SymmetryGroup(U1(:N), U1(:twoSz))
q(n, s) = sector(G; N = n, twoSz = s)
site = Leg(Dict(q(0, 0) => 1, q(1, 1) => 1, q(1, -1) => 1, q(2, 0) => 1);
name = :site)
bath = Leg(Dict(q(0, 0) => 2, q(1, 1) => 3, q(1, -1) => 3); name = :bath)
F = fusionmap(site, bath)
println("product leg: ", F.out)
rng = MersenneTwister(1)
X = randtensor(rng, Float64, (F.out, dual(F.out)), neutral(labeltype(G)))
H = X + X'
vals, U = eigen(H)
kept, rec = global_cut(vals; nkeep = 6, gaptol = 1e-10)
println("kept per sector: ", Dict(s.label.q => k for (s, k) in kept if k >
```

```
0), "  record: ", rec)
A = split_legs(U, 1, F)
println("isometry legs: ", [TNOps.legname(l) for l in legs(A)], ", ",
length(blockkeys(A)), " blocks")
I1 = contract(A, (1, 2), adjoint(A, (1, 2), (3,)), (2, 3))
@printf("max |A†A - 1| = %.1e\n", maximum(abs.(todense(I1) - I)))
```

```
product leg: Leg([Out; Sector(0, 0, 0)=>2, Sector(1, -1, 1)=>5, Sector(1, 1,
1)=>5, Sector(2, -2, 0)=>3, Sector(2, 0, 0)=>8, Sector(2, 2, 0)=>3, Sector
(3, -1, 1)=>3, Sector(3, 1, 1)=>3])
kept per sector: Dict((1, -1, 1) => 1, (2, 2, 0) => 1, (2, 0, 0) => 2, (1,
1, 1) => 1, (0, 0, 0) => 1)  record: (requested = 6, actual = 6, gap = 0.508
8442359284979, cut_moved = false, total = 32)
isometry legs: [:bath, :site, :bond], 12 blocks
max |A†A - 1| = 1.5e-15
```

In [9]:
```
println("Summary (largest deviation per check):")
for (k, v) in sort(collect(RESULTS); by = first)
    @printf("  %-24s %.1e\n", k, v)
end
```

```
Summary (largest deviation per check):
  apply_parity             0.0e+00
  arithmetic               4.0e-16
  contract                 3.1e-16
  dense round trip         0.0e+00
  eigen                    2.4e-15
  eigen (partition)        1.9e-15
  forbidden output blocks  0.0e+00
  fuse ∘ split             0.0e+00
  ncon                     2.4e-16
  parity_tensor            0.0e+00
  permute                  0.0e+00
  qr                       2.6e-16
  qr (partition)           3.5e-16
  svd                      6.7e-16
  svd (partition)          2.6e-15
  tr                       1.0e-16
```